

Summary of Findings

Analysis of the Atlas Sphere code shows several **security and scalability issues** in the atomic swap components:

- **Weak HTLC Secret Generation:** The function `HtlcSecret::generate()` in `cross-vm-coordinator` uses only the current time and process ID to seed a SHA-256 hash ¹. This is not cryptographically secure. The code even notes *“production: use OsRng”*. Secrets should instead be drawn from a secure random source (e.g. `rand::rngs::OsRng`) to prevent predictability.
- **Inefficient Nonce Tracking:** In the `CrossVmBridge` (kernel pallet), a replay protection list (`used_nonces`) is stored as a `Vec<u64>`. Checking or inserting into a vector is **O(n)**, which will not scale as the list grows. For production, this should be replaced by a `HashSet<u64>` (or `BTreeSet`) to allow O(1) average-time lookups.
- **Missing Session Count Methods:** The integration tests for `SwapCoordinator` refer to methods like `pending_count()` and `completed_count()`, but these methods are not implemented. Without them, users of the coordinator cannot query how many sessions are pending or completed, breaking tests and any status monitoring.
- **No HTLC Replay Nonce Enforcement:** The coordinator does not enforce a per-HTLC unique nonce beyond the session scope. There should be a global nonce or tracking to ensure a revealed secret or reuse cannot be replayed across sessions. As-is, an adversary might reuse an old HTLC claim if it isn't explicitly prevented.
- **No Session Limits:** `SwapCoordinator` lacks a limit on the number of concurrent sessions. In high-load or DoS scenarios, this could lead to unbounded memory growth or resource exhaustion.
- **Incomplete `validate_unsigned` Logic:** The kernel pallet's off-chain function `record_flash_finality_anchor` is missing in the `validate_unsigned` implementation. This means unsigned transactions calling that function would be rejected, potentially breaking any flows relying on it.
- **Missing Weight Benchmarks:** The `x3-atomic-kernel` pallet lacks benchmarks (no functions annotated under `#[cfg(feature = "runtime-benchmarks")]`). Without weights, gas costs are not estimated, which is required for mainnet readiness.
- **No Dead-Session Cleanup:** Once a swap session is created, there is no mechanism to remove or time out “stuck” sessions. Over time, old sessions will accumulate in memory/storage, causing a leak.

- **Dispatcher Not Used in Cross-VM Bridge:** The `cross-vm-bridge::execute_operation` function does not utilize the runtime's `CrossVmDispatcher` trait. It currently returns a stubbed result rather than actually dispatching the payloads to the EVM/SVM adapters.
- **Missing Cryptographically Secure RNG:** As noted, `HtlcSecret::generate()` should use `OsRng` or similar. Its current approach (time+PID hashing) is insufficient and was likely chosen only for demo purposes ¹.

Each of these issues is explained in detail below, with code snippets and references.

Weak HTLC Secret Generation

The `HtlcSecret::generate()` function creates a secret by hashing the current system time and process ID ¹:

```
pub fn generate() -> Self {
    let mut rng_bytes = [0u8; 32];
    // Use system time + process entropy as seed (production: use OsRng)
    let now =
        SystemTime::now().duration_since(UNIX_EPOCH).unwrap_or_default().as_nanos();
    let mut hasher = Sha256::new();
    hasher.update(now.to_le_bytes());
    hasher.update(b"x3-htlc-secret");
    hasher.update(process::id().to_le_bytes());
    let hash = hasher.finalize();
    rng_bytes.copy_from_slice(&hash);
    Self(rng_bytes)
}
```

This method is **not cryptographically secure**. An attacker who can guess the approximate time and knows the process ID could predict the secret. The comment itself indicates this should be replaced for production.

Remediation: Replace this with a secure RNG, e.g. using the `rand` crate's `OsRng`:

```
use rand::rngs::OsRng;
use rand::RngCore;

pub fn generate() -> Self {
    let mut rng = OsRng;
    let mut secret = [0u8; 32];
    rng.fill_bytes(&mut secret);
    Self(secret)
}
```

This change ensures the secret is unpredictable (entropy from the OS).

Inefficient Nonce Tracking

The kernel pallet (in `x3-kernel`) or the cross-VM bridge uses a list of used nonces to prevent replay. If implemented as a `Vec<u64>`, every check `if used_nonces.contains(&nonce)` is $O(n)$. As the number of records grows, performance will degrade.

Issue: The user message specifically calls out “**CrossVmBridge::used_nonces is a `Vec<u64>` — $O(n)$ replay protection**”.

Remediation: Change the storage type to a `HashSet<u64>` (or `BTreeSet`). For example, in storage:

```
#[pallet::storage]
pub type UsedNonces<T> = StorageMap<_, Blake2_128Concat, u64, (), ValueQuery>;
```

Then check with `UsedNonces::contains_key(nonce)` ($O(1)$ average). All insertions and removals become efficient, allowing scalability.

Missing `pending_count()` and `completed_count()`

The integration tests refer to `pending_count()` and `completed_count()` methods on the swap coordinator, but the code (likely in `SwapCoordinator` or its underlying storage) does not implement them. For example, a test might do:

```
assert_eq!(coordinator.pending_count(), some_expected_value);
```

But since these functions don't exist, the tests will not compile or will fail.

Remediation: Add these methods, counting active and finished sessions. For example, if sessions are stored in a map, count entries by state:

```
impl SwapCoordinator {
    pub fn pending_count(&self) -> usize {
        self.sessions.iter().filter(|(&_, s)| s.phase !=
SwapPhase::Complete).count()
    }
    pub fn completed_count(&self) -> usize {
        self.sessions.iter().filter(|(&_, s)| s.phase ==
SwapPhase::Complete).count()
    }
}
```

Adjust as needed based on how sessions are stored. Update tests if their semantics differ.

HTLC Replay Nonce Protection

Currently, once a secret is used to claim an HTLC in one session, the code does not globally mark that nonce used. An attacker could potentially reuse the same secret/nonce in another session. The **"replay nonce"** should be unique chain-wide or at least per-coordinator instance.

Remediation: Track globally (or in coordinator config) used secrets or nonces for HTLCs. For each `SwapCoordinator`, keep a `HashSet<HtlcHash>` of revealed secrets. Before allowing a claim, check that the hash hasn't been used. Upon a claim (fast or slow), insert the secret's hash into this set. This prevents replay across sessions.

Concurrent Session Limit

The `SwapCoordinator` currently has no limit on how many swap sessions can be active at once. In heavy use, this could exhaust memory.

Remediation: Add a configurable max sessions limit to the coordinator. In `CoordinatorConfig`, add a field like `max_sessions: usize`. In `setup_swap()`, check `if self.sessions.len() >= config.max_sessions { return Err(...) }`. This ensures the coordinator rejects new sessions when resources are bounded.

Missing `validate_unsigned` for `record_flash_finality_anchor`

In the pallet (`x3-kernel`), if there is an off-chain or unsigned extrinsic named `record_flash_finality_anchor`, the `validate_unsigned()` implementation must allow it. The user hint suggests that it's missing coverage, meaning currently such a call might be rejected.

Remediation: In the pallet's `Config`, ensure `validate_unsigned` includes this call. For example:

```
#[pallet::validate_unsigned]
fn validate_unsigned(source: TransactionSource, call: &Call<T>) ->
TransactionValidity {
    if let Call::Kernel(pallet::Call::record_flash_finality_anchor { .. }) =
call {
        return ValidTransaction::with_tag_prefix("record_flash_finality_anchor")
            .priority(T::UnsignedPriority::get())
            .and_provides(...).build();
    }
    InvalidTransaction::Call.into()
}
```

Fill in validity details as needed. This ensures the runtime accepts the unsigned transaction.

Weight Benchmarks Missing

The pallet's `benchmarking` module is empty or not configured. Without benchmarks, weights cannot be auto-generated, which is required by frame.

Remediation: Use Substrate's benchmarking framework. Annotate functions with `#[pallet::benchmark]` and implement weight measurements. Run `cargo benchmark` to generate the `weights.rs` file. This is a straightforward task but requires writing benchmarks that simulate each extrinsic.

Dead-Session Cleanup (Memory Leak)

Sessions that never complete (e.g. due to crashes or user abandonment) remain in the coordinator's memory. Over time, this leaks.

Remediation: Implement a cleanup mechanism. For example, each session could have an expiration time (`created_at + TTL`). Periodically (or on each new request), scan and remove expired sessions. Alternatively, use a background thread or worker to garbage-collect. If using on-chain pallet, sessions might auto-expire after a certain number of blocks.

Cross-VM Bridge `execute_operation` Stub

The `execute_operation` in `x3_cross_vm_bridge` currently likely just returns a dummy result. Instead, it should use the configured `CrossVmDispatcher` to actually execute the payload. For example:

```
let result_evm = T::EvmAdapter::execute_evm(evm_payload);
let result_svm = T::SvmAdapter::execute_svm(svm_payload);
```

Replace stubs with real dispatcher calls so that cross-VM calls truly run on both VMs.

Cryptographically Secure RNG

As noted, replace the custom SHA-256 approach in `HtlcSecret::generate()` with a secure RNG. Use `rand::rngs::OsRng` or similar (citing [89†L37-L45] as the current implementation and comment).

Summary of Prioritized Fixes

1. **HTLC Secret RNG:** Replace with `OsRng` ¹ (critical for security).
2. **Nonce Tracking:** Change `used_nonces` to `HashSet` (critical for scalability).
3. **Session Methods:** Implement `pending_count()` / `completed_count()` (tests will fail otherwise).

4. **Replay Protection:** Add global nonce/secret tracking in coordinator (security).
5. **Session Limit:** Enforce `max_sessions` (robustness).
6. **Validate Unsigned:** Add `record_flash_finality_anchor` to `validate_unsigned` (functionality).
7. **Benchmarks:** Write pallet benchmarks (deployment readiness).
8. **Session Cleanup:** Implement TTL or GC for old sessions (resource cleanup).
9. **Dispatcher Calls:** Fix `execute_operation` to use actual dispatcher (functionality).

For each item, code patches would involve updating the respective files (e.g., `types.rs`, the pallet's lib, the coordinator logic). In tests, add cases verifying RNG uniqueness (already present), nonces can't be reused, and that the pending/completed counts match expectations.

Sources: The primary source is the Atlas Sphere repository itself (shown above) ¹. The identified issues come from the code review of this repo's crates and pallets. Where applicable, standard Rust/crypto best practices were applied.

¹ `types.rs`

<https://github.com/Cytopimpinainteazy/atlas-sphere/blob/5b4284c238c512de98361038cabf6b6ac99d664e/crates/cross-vm-coordinator/src/types.rs>